Representing the SKI Combinator Calculus in the Nock Instruction Set Architecture

N. E. Davis ~lagrev-nocfep
ZeroAttest, Groundwire Foundation

Abstract

The Nock ISA opcodes 2, 1, and 0 have been said to correspond to the combinators S, K, and I in the SKI combinator calculus. We give a four-rule compiler from arbitrary SKI terms to Nock formulas using only opcodes 0, 1, and 2, prove it correct under a fixed application convention, and exercise it on the usual worked examples: the identity laws, Church booleans, the B, C, and W combinators, and Church numerals performing real arithmetic. The compiled output is verified against an independent Nock 4K interpreter. The construction is a constructive proof that the Nock fragment of 0, 1, 2 opcodes is Turing-complete. This fragment lacks certain properties, marking a boundary beyond which the fragment is genuinely insufficient and additional tools are required.

Contents

Manuscript submitted for review.

Address author correspondence to ~lagrev-nocfep.

1 Introduction

The Nock instruction set architecture is a combinator calculus sufficient to represent all computable functions. It has been a longstanding temptation to draw a direct analogy from the Nock ISA to the `SKI` combinator calculus. This analogy claims that the Nock opcodes 2, 1, and 0 correspond to the combinators `S`, `K`, and `I` respectively. The analogy is usually stated informally, e.g. "[there are] some subtle differences to Nock's expression of `S` as opcode 2 that we will elide as being fundamentally similar, but perhaps worthy of its own monograph" (`~lagrev-nocfep` and `~sorreg-namtyv`, 2025). We here deliver upon that promise and convert an Urbit legend into a concrete demonstration.

Combinator calculi build all computation from a handful of variable-free operators; the `SKI` system does it with three (Curry and Feys, 1958), which is neither unique nor minimal but balances convenience with economy.[1] Put briefly, the `S` combinator ("substitute") takes three arguments and applies the first to the third, then applies the second to the third, and finally applies the result of the first application to the result of the second. The `K` combinator ("'k'onstant") takes two arguments and returns the first, discarding the second. The `I` combinator ("identity") takes one argument and returns it unchanged. The `SKI` system is Turing-complete, meaning that any computable function can be expressed as a combination of these three operators.

```
S x y z = x z (y z)
K x y   = x
I x     = x
```

The Nock ISA is also Turing-complete, and it is natural to ask whether the `SKI` combinators can be represented in Nock. The answer is yes, but some details are subtle. The analogy between Nock opcodes and `SKI` combinators is not a simple one-to-one compilation procedure, but a homomorphism that preserves the structure of the computation.

A straightforward reading of Nock opcode `0` yielding `I` is uncontroversial, as `[0 1]` trivially returns identity. The reading of opcode `1` yielding `K` is also straightforward, as `[1 x]` returns a constant function that discards its argument (subject). The reading of opcode `2` yielding `S` is more subtle, as it requires a fixed convention for how the compiled combinator receives its arguments. The "subtle differences" of `~lagrev-nocfep` and `~sorreg-namtyv`'s elision matter more than they appear. The opcode-to-combinator analogy is not a compilation procedure. SKI terms are curried and higher-order: combinators are routinely partially applied, and a combinator may be passed as an argument to another combinator. A symbol-for-symbol substitution—writing `2` for `S`, `1` for `K`, and `0` for `I`,

[1] See particularly Smullyan (1994) for more on combinatory logic.

with appropriate transposition of arguments into subject and formula—produces nothing runnable the moment `S` has fewer than three arguments, which is almost always. What is needed is a *homomorphism*: a translation under which Nock application of compiled terms mirrors SKI reduction, with a fixed convention for how a compiled combinator receives its arguments.

We supply such a homomorphism as four rules with three small Nock formulas yielding the `SKI` combinators. The remainder of this article consists of worked examples, mechanical validation, a cost analysis, and an accounting of the limitations.

2 Preliminaries

We assume Nock 4K. Evaluation is the function `*[subject formula]`. We require only three opcodes from the Nock ISA:

```
:: slot: fetch the noun at tree address b
*[a 0 b]    ->  /[b a]

:: quote: return b unchanged
*[a 1 b]    ->  b

:: eval: compute a subject and a formula, then run
*[a 2 b c]  ->  *[*[a b] *[a c]]
```

plus the structural distribution (autocons) rule, which fires whenever a formula's head is itself a cell:

```
*[a [b c] d]  ->  [*[a b c] *[a d]]
```

Slot address 1 denotes the whole subject, so `*[a 0 1] = a`.

A *value* is a Nock formula that consumes its argument as the *subject*. To apply a value `v` to an argument `x`, we evaluate

```
apply(v, x)  ==  *[x v]
```

This operative convention inverts the lambda-calculus argument order, because in Nock the function is the formula and the argument is the subject.

3 Combinator Formulas

Under the convention of Section 2, each `SKI` combinator is a closed Nock formula. We give each, then verify its defining law by reduction.

3.1 `I` identity

`I x = x` requires `*[x I] = x`, which is slot 1:

```
I = [0 1]
```

3.2 `K` constant

Applying `K` to `x` must yield a value that ignores its next argument and returns its value. Thus `K x y = x` requires `*[x K] = [1 x]`, which is built from subject `x` by autocons:

```
K = [[1 1] [0 1]]
```

For example,

```
*[x K] = [*[x [1 1]] *[x [0 1]]] = [1 x]
```

Then for any `y`, `*[y [1 x]] = x`, so `K x y = x` holds.

3.3 `S` substitution

Substitution states `S x y z = (x z)(y z)`. The combinator accumulates `x`, then `y`, then fires on `z`. We construct it bottom-up.

When the fully-applied `S x y` finally receives `z` as subject, it must produce `(x z)(y z)`. Under the convention, `x z = *[z x]` and `y z = *[z y]`, and the outer application is `*[ *[z y] *[z x] ]`. That is precisely opcode 2 with subject `z`, formula-`b` equal to `y`, formula-`c` equal to `x`:

```
S x y = [2 y x]
::  *[z [2 y x]] = *[*[z y] *[z x]] = (x z)(y z)
```

Working back one step, applying `S x` to `y` (subject `y`, constant `x`) must build the noun `[2 y x]`:

```
S x = [[1 2] [0 1] [1 x]]
```

We check that `*[y S x] = [2 y x]`, since `*[y [1 2]] = 2`, `*[y [0 1]] = y`, and `*[y [1 x]] = x`. Working back the final step, applying `S` to `x` (subject `x`) must build `S x`; the first two parts are constants and the third, `[1 x]`, is built from `x` by the same autocons trick used for `K`:

```
S = [[1 [1 2]] [1 [0 1]] [[1 1] [0 1]]]
```

We check the evaluation: `*[x S] = [[1 2] [0 1] [1 x]] = S x`.

3.4 `B` composition

Any other combinator can be reached by elaborating its lambda definition to `SKI` and compiling, but the result is large. A named combinator is better derived directly, by the same method as the previous subsection: it becomes one builder layer per argument. The innermost layer is the opcode 2 application skeleton that fires on the final argument; each enclosing layer is an autocons that captures one argument and embeds it. We here derive `B`, `C`, and `W`; a similar methodology could be used for much of Smullyan's aviary (Smullyan, 1994).

For `B f g x = f (g x)`, the firing step computes `f` applied to `g x`, i.e. `*[*[x g] f]`, which is opcode 2 with `g` in the formula-`b` slot and `f` quoted in the formula-`c` slot; capturing `g` then `f` gives:

```
::  *[x B f g] = *[*[x g] f] = f (g x)
B f g = [2 g [1 f]]
::  *[g B f]   = [2 g [1 f]]
B f   = [[1 2] [0 1] [1 [1 f]]]
::
B     = [[1 [1 2]] [1 [0 1]] [[1 1] [[1 1] [0 1]]]]
```

3.5 `C` transposition

The same procedure yields `C` (flip):

```
::  f x y -> f y x
C = [[1 1 2] [1 [1 1] 0 1] [1 1] 0 1]
```

Table 1: Compiled Nock cell counts for various `SKI` terms, using direct derivation rather than bracket abstraction; compare to Table 3.

Combinator	Direct	Via `SKI`	Ratio
`B`	11	208	≈19×
`C`	11	208	≈19×
`W`	6	130	≈22×

3.6 `W` duplicate

Likewise, `W` (duplicate):

```
::  f x   -> f x x
W = [[1 2] [1 0 1] 0 1]
```

3.7 Commentary

The one subtlety is how a captured argument is embedded, which is dictated by how the combinator uses that argument. If the argument is applied to a computed value (as `B`'s `f` is applied to `g x`), it occupies a formula slot and is embedded quoted, `[1 f]`. If it is applied to the current subject (as `W`'s `f` is applied to `x`) it must pass through as the live formula, embedded by slot, `[0 1]`. Quote where the argument is data; slot where the argument is the active function.

Notably, the value forms of `B`, `C`, and `W` use only Nock opcodes 0 and 1. The opcode 2 appears only in the formula they assemble at application time (`[2 g [1 f]]`), carried as quoted data. The combinators are pure construction; the application machinery is the structure they emit. Among the basis only `S` carries a 2 in its own body, because it performs two applications at once.

The size payoff over bracket abstraction is large (cell counts; see Section 7 for method), as recorded in Table 1.

The two forms are extensionally identical on every input tested. For any combinator known by name, direct derivation is preferred and the abstraction tax is avoided entirely; bracket abstraction remains the general fallback for arbitrary lambda terms.

4 Compilation of the Homomorphism

Let `[[t]]` denote a *closed* Nock formula whose product is the value of the `SKI` term `t`. The compiler consists of four rules:

```
[[S]]   = [1 S]
[[K]]   = [1 K]
[[I]]   = [1 I]
[[A B]] = [2 [[B]] [[A]]]
```

with `S`, `K`, `I` the formulas of Section 3. The combinators are *quoted* so that `[[·]]` always denotes a value-producing computation; application is opcode 2 applied to the two compiled operands. `[[t]]` produces the value of `t`; it is closed, so the naïve approach of `*[a [[t]]]` yields `val(t)` for any subject `a` and is not yet an application. To apply `t` to Nock-noun arguments `a[0] ... a[n]`, fold them onto the value, each quoted, by opcode 2:

```
run(t; a[1] ... a[n]) = *[0 F[n]]
```

where `F[0] = [[t]]`, `F[i] = [2 [1 a[i]] F[i-1]]`. Each layer `*[· [2 [1 a[i]] F]]` reduces to `*[a[i] val]`, the value applied to `a[i]` as subject. Throughout the remainder of this paper, we will write `t a[1] ... a[n] = r` as shorthand for `run(t; a[1] ... a[n]) = r`. The arguments are Nock nouns, not `SKI` terms: atoms in the identity and boolean laws, real formulas such as `+/INC` in the arithmetic examples. A tap must not fire until its formula is a concrete noun.

Why quote, and why this order? Evaluating `[[A B]] = [2 [[B]] [[A]]]` against any subject `s` gives `*[*[s [[B]]] *[s [[A]]]]`. Because `[[B]]` and `[[A]]` are closed, they ignore `s` and reduce to the values `val(B)` and `val(A)`. The result is `*[val(B) val(A)]`—apply value `val(A)` to argument `val(B)`, i.e. `A` applied to `B`. The operands are reduced to values *before* application, which is what makes nested applications compose correctly; quoting a sub-application rather than evaluating it produces an "off-by-one error" (a residual `K` where the value was expected).

Correctness is demonstrated practically by induction on term structure. For application, the rule computes `apply(val(A), val(B))`, and by the induction hypothesis `val(A)`, `val(B)` are the values of `A`, `B`; by §2 this is the value of `A B`. The translation is therefore a homomorphism from `SKI` application to Nock evaluation, under the call-by-value strategy forced by opcode 2's eager semantics (see Section 8).

5 Worked Examples

All terms below are written in lambda form for legibility, elaborated to `SKI` by standard bracket abstraction (Kiselyov, 2018), then compiled by the Nock `SKI` compiler and run. The results are the products actually computed.

A script which resolves the `SKI` terms to Nock formulas, runs them in a Nock 4K interpreter, and checks the results is available at `sigilante/skitrace`.

5.1 Identity laws

From canonical `SKI`, we anticipate the following identity laws to hold true:

```
I 42        = 42
S K K 42    = 42          :: S K K → I
S K S 42    = 42          :: S K * → I
```

The evaluation of `S K K 42`, worked in Nock `SKI`, is shown in Listings 1, 2, and 3. The process is mechanical and discursive. The evaluation of `S K S 42` is similar and left as a proverbial exercise to the reader.

Listing 1: Compilation expansion of `S K K 42`.

```
*[42 [[S K K]]]
*[42 2 [[K]] [[S K]]]
                             [[A B]] = [2 [[B]] [[A]]]
*[42 2 [1 K] [[S K]]]
                             [[K]] = [1 K]
```

```
*[42 2 [1 K] 2 [[K]] [[S]]]
                         [[A B]] = [2 [[B]] [[A]]]
*[42 2 [1 K] 2 [1 K] [[S]]]
                         [[K]] = [1 K]
*[42 2 [1 K] 2 [1 K] 1 S]
                         [[S]] = [1 S]
*[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
    1] 0 1]
                         substitute S,K,I formulas
```

Listing 2: Evaluation of `S K K 42`.

```
*[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
    1] 0 1]
                Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
*[*[42 1 [1 1] 0 1] *[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0
    1] [1 1] 0 1]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] *[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
    1] 0 1]]
                Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
*[[[1 1] 0 1] *[*[42 1 [1 1] 0 1] *[42 1 [1 1 2] [1 0 1]
    [1 1] 0 1]]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] *[[[1 1] 0 1] *[42 1 [1 1 2] [1 0 1] [1 1]
    0 1]]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] *[[[1 1] 0 1] [1 1 2] [1 0 1] [1 1] 0 1]]
                cons  *[a [b c] d] = [*[a b c] *[a d]]
*[[[1 1] 0 1] *[[[1 1] 0 1] 1 1 2] *[[[1 1] 0 1] [1 0 1]
    [1 1] 0 1]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] [1 0 1] [1 1] 0 1]]
                cons  *[a [b c] d] = [*[a b c] *[a d]]
*[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] 1 0 1] *[[[1 1] 0 1]
    [1 1] 0 1]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] [1 1] 0 1]]
                cons  *[a [b c] d] = [*[a b c] *[a d]]
*[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] 1 1] *[[[1 1] 0
    1] 0 1]]
                Nock 1  *[a 1 b] = b
*[[[1 1] 0 1] [1 2] [0 1] 1 *[[[1 1] 0 1] 0 1]]
```

```
                   Nock 0  *[a 0 b] = /[b a]
*[[[1 1] 0 1] [1 2] [0 1] 1 /[1 [[1 1] 0 1]]]
                   /[1 a] = a
*[[[1 1] 0 1] [1 2] [0 1] 1 [1 1] 0 1]
                   cons  *[a [b c] d] = [*[a b c] *[a d]]
[*[[[1 1] 0 1] 1 2] *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
                   Nock 1  *[a 1 b] = b
[2 *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
                   cons  *[a [b c] d] = [*[a b c] *[a d]]
[2 *[[[1 1] 0 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
                   Nock 0  *[a 0 b] = /[b a]
[2 /[1 [[1 1] 0 1]] *[[[1 1] 0 1] 1 [1 1] 0 1]]
                   /[1 a] = a
[2 [[1 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
                   Nock 1  *[a 1 b] = b
[2 [[1 1] 0 1] [1 1] 0 1]
(= [2 K K], the identity combinator's value)
```

Listing 3: Application of `S K K 42`.

```
# applying val(S K K) to 42   ( *[42 val(S K K)] )
val(S K K) = [2 [[1 1] 0 1] [1 1] 0 1]

# evaluation of *[42 [2 [[1 1] 0 1] [1 1] 0 1]]
*[42 2 [[1 1] 0 1] [1 1] 0 1]
                   Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
*[*[42 [1 1] 0 1] *[42 [1 1] 0 1]]
                   cons  *[a [b c] d] = [*[a b c] *[a d]]
*[[*[42 1 1] *[42 0 1]] *[42 [1 1] 0 1]]
                   Nock 1  *[a 1 b] = b
*[[1 *[42 0 1]] *[42 [1 1] 0 1]]
                   Nock 0  *[a 0 b] = /[b a]
*[[1 /[1 42]] *[42 [1 1] 0 1]]
                   /[1 a] = a
*[[1 42] *[42 [1 1] 0 1]]
                   cons  *[a [b c] d] = [*[a b c] *[a d]]
*[[1 42] *[42 1 1] *[42 0 1]]
                   Nock 1  *[a 1 b] = b
*[[1 42] 1 *[42 0 1]]
                   Nock 0  *[a 0 b] = /[b a]
*[[1 42] 1 /[1 42]]
                   /[1 a] = a
*[[1 42] 1 42]
```

```
                  Nock 1  *[a 1 b] = b
42
```

5.2 Church booleans

Conventional Nock utilizes atoms, but `SKI` combinators are pure construction; we therefore must utilize Church-style booleans and numerals within the particular three-opcode discipline we follow here. Thus while this section is demonstrative of `SKI` in Nock, it evades the usual Nock idioms and in particular the economy afforded by the introduction of opcodes 3, 4, and 5.

A Church boolean is a two-argument function that selects one of its arguments. Because the combinators are pure construction, the truth values are instead represented as combinator sequences. With `TRUE = K` and `FALSE = K I`:

```
TRUE p q
  = K p q
  = p

FALSE p q
  = K I p q
  = I q
  = q

TRUE  7 8 = 7
FALSE 7 8 = 8
```

These are the canonical encodings from `SKI` praxis; here we probe with atom arguments because neither combinator applies its arguments as functions.

5.3 The B, C, W combinators

Elaborated from their defining lambda terms:

```
B = λ f g x. f (g x)    compose
C = λ f x y. f y x      transpose
W = λ f x.   f x x      duplicate
```

Driving B with a real Nock increment `inc = [4 0 1]` as payload, and `C`, `W` with atoms:

```
B inc inc 7  = 9       :: inc (inc 7)
C K 7 8      = 8       :: K 8 7 = 8
W K 7        = 7       :: K 7 7 = 7
```

These exercise three-variable abstraction and argument duplication. Note that the compiled `SKI` scaffolding remains pure 0, 1, 2; the only 4 in `B`'s run is the external increment supplied as data.[2]

5.4 Church numerals and arithmetic

A Church numeral `church n` applies its first argument `n` times to its second. Numerals are built and incremented entirely within the fragment.[3] Examples of Church numerals and their Nock `SKI` representations are given in Table 2. At this point, we will revert in part to the lambda calculus notation for clarity, but the underlying Nock `SKI` formulas are always available and are used in the actual evaluation. The successor function is defined by the usual combinator formula:

```
church n = λλf.x. f^n x
succ = λλλn.f.x. f (n f x)

succ (church 0) = church 1
succ (church 1) = church 2
```

In explicit Nock `SKI` form, the Church numeral successor function becomes:

```
succ = [[1 1 2] [0 1] 1 [1 1] 0 1]

church 2 = [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
```

[2] We here draw a sharp distinction between opcode 4 as `inc = [4 0 1]` which operates on Nock atoms, and `succ` which operates on Church numerals. The two are distinct: `succ` maps a numeral-value to a numeral-value in 0,1,2, while `inc` maps an atom to an atom via opcode 4.

[3] A Church numeral is a value (formula), not an atom. To read it out as a Nock atom we decode it: apply it to the atom successor `inc = [4 0 1]` and the atom `0`, writing `⌊n⌋= n inc 0`: `⌊church 0⌋= 0 ; ⌊SUCC^5 (church 0)⌋= 5 ; ⌊SUCC (church 2)⌋= 3`. Decoding is the only step that uses opcode 4, and it is readout, not arithmetic.

```
             [1 1] 0 1]
⌊church 2⌋ = 2
succ (church 2) = [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
                  [1 1] 0 1] [1 1] 0 1]
⌊succ (church 2)⌋ = 3
```

Arithmetic over the Church numerals is defined by the usual combinator formulas:

```
::  in Church numerals
plus = λλλλm.n.f.x. m f (n f x)
mult = λλλm.n.f. m (n f)

::  alternatively
plus = λm n. m succ n
mult = λm n. m (plus n) (church 0)

::  compiled in Nock SKI form
plus = [[1 1 1 2] [1 0 1] [1 1 1 1] [1 1] 0 1]
mult = [[1 1 2] [1 0 1] [1 1 1] [1 1] 0 1]

::  in Nock decoding
⌊church 0⌋ = 0
⌊church 3⌋ = 3
⌊church 5⌋ = 5
⌊plus 2 3⌋ = 5
⌊mult 3 4⌋ = 12
```

Table 2: Church numerals and their Nock `SKI` representations. The numeral `church n` is a value (formula) that applies its first argument `n` times to its second. The decoding `⌊n⌋= n inc 0` produces the Nock atom corresponding to the numeral.

`n`	`⌊church n⌋`	`church n`
0	0	`[1 0 1]`
1	1	`[[1 2] [1 0 1] [1 1] 0 1]`
2	2	`[[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1]`
3	3	`[[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]`
4	4	`[[1 2] [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]`
5	5	`[[1 2] [[1 2] [[1 2] [[1 2] [[1 2] [1 0 1][1 1] 0 1] [1 1] 0 1] [1 1] 0 1]` `[1 1] 0 1] [1 1] 0 1]`

Table 3: Compiled Nock cell counts for various `SKI` terms. These numbers are via bracket abstraction; compare to Table 1.

Term	`SKI` leaves	Nock cells
`I`	1	2
`S K K`	3	22
`W`	16	130
`B`	25	208
`C`	25	208
`church 2`	76	640
`church 5`	187	1588
`church 10`	372	3168

6 Validation

Any valid Nock ISA interpreter should correctly evaluate the examples above after the `SKI` compiler stage has been applied. We utilized both a simple interpreter with only opcodes 0, 1, and 2,[4] and the `pinochle` interpreter with the full Nock ISA.[5] Results were identical (and correct) in both cases. The examples are not exhaustive, but they exercise the combinators and the compiler in a variety of ways, including nested applications, argument duplication, and Church-style numerals and booleans.

7 Cost

The transformation is faithful but not frugal. Measuring `SKI` leaf count against compiled Nock cell count shows the extreme verbosity of the latter. Table 3 shows the compiled size of several terms, including the Church numerals and the named combinators.

Compiled Nock runs roughly $8.5\times$ the `SKI` leaf count (a constant per combinator plus the opcode 2 application wrapper) and the `SKI` itself is already exponential in the worst case

[4] `sigilante/skitrace`

[5] `sigilante/pinochle`.

under naive bracket abstraction. The natural number ten compiles to a 3168-cell formula as a Church numeral. These figures are static, before reduction; at runtime `S` duplicates its argument into both branches with no sharing, so evaluation cost compounds on representation cost.

Named combinators escape this blow-up before the compiler stage: derived directly rather than routed through bracket abstraction, `B`, `C`, and `W` are roughly twenty times smaller. The cost above is the price of the *general* compiler on arbitrary terms, not a property of the targets themselves.

8 Limitations

Three properties bound the construction. Each is a real boundary, not a deficiency to be patched away within the fragment.

1. **Call-by-value.** Opcode 2 is eager: `*[a 2 b c]` fully reduces `*[a b]` and `*[a c]` before applying. The compilation is therefore applicative-order. A term whose normal form depends on discarding a divergent subterm will loop. Concretely, with `ω = S I I` and `Ω = ω ω`, normal-order `K I Ω` reduces to `I`, but the compiled form diverges. For a compilation target this is usually acceptable; for a faithful lazy surface it is not, and recovering normal order requires explicit thunking, which reintroduces runtime branching and pulls in opcodes 3 and 6.

2. **No readback in {0, 1, 2}.** Compiled `SKI` can be executed but its normal form cannot be printed without distinguishing a residual `S` from a `K` from an application at runtime. For Nock, this requires a structural test (opcode 3), an equality test (opcode 5), and a conditional (opcode 6). The fragment is a faithful execution target, not a normalizer.

   A normalizer must additionally return the normal form in an inspectable form, which means recognizing combinators at runtime, which the {0, 1, 2} fragment cannot do. In other words, when an `SKI` evaluator completes, it has a normal form or a tree whose leaves are `S`, `K`, `I` and whose internal nodes are applications. To print that normal form (to "read it back"), one walks the tree and at each node decides if it is a leaf or an application, and if a leaf, which combinator?

3. **No sharing.** This is tree reduction. `S` copies its argument with no sharing, so terms with shared redexes blow up exponentially. This is moderately acceptable as an intermediate representation to compile through, but impractical as a runtime for large terms, which want graph reduction.

These can be read as an alternate set of motivations for opcodes beyond 2; as may be surmised, the {0, 1, 2} fragment is Turing-complete but lacks certain practical features.

9 Conditionals

A conditional is an obvious stress test for such a small fragment because in a strict evaluator a conditional that evaluates both branches is useless for its principal job of guarding recursion. In practice, the conditional must be split into two parts.

The Church booleans are already a form of conditional; with `TRUE = K` and `FALSE = K I`, the term `b t e` routes to the chosen branch. The routing itself executes neither branch: `K` reads its two arguments by slot and discards one, never applying them as formulas. If the branches are held as quoted formulas (thunks) and only the survivor is run, the unchosen branch is never evaluated. The whole conditional is one closed formula in the fragment:

```
IF cond tf ef s
             = [2 [1 s] [2 [1 ef] [2 [1 tf] [1 cond]]]]
```

Reading inner to outer: apply `cond` to `tf`, then to `ef` (selection), then run the survivor on subject `s`. It is fifteen cells, pure {0, 1, 2}.

This conditional is genuinely lazy.[6] The discipline this requires is exact: branches must be kept as quoted formulas rather than compiled as combinator sub-terms. Compile a divergent branch and the eager opcode 2 forces it before selection ever happens (the `K I Ω` divergence). Thunking the branch keeps it inert until forced; this is similar to stored procedures in conventional Nock using cores with opcode 9.

What the `SKI`/{0, 1, 2} fragment cannot do effectively is manufacture the boolean. `IF` above routes on a boolean it is handed; to branch on a property of data ("is this atom zero", "is this noun a cell"), one must first map a datum to `K` or `K I`, and that inspection reads an atom's value or tests cell-ness. Neither is expressible in pure {0, 1, 2}. Selecting on a given boolean is free; deciding what the boolean should be requires the cell test (opcode 3), increment, equality (opcode 5), or negation (built with opcode 6 on opcode 5).

This is precisely the shape of Nock's own conditional. Opcode 6 is trivially derivable from opcodes 0–5, and what it adds over the selector above is the data-derived predicate: run a formula `b` to compute a loobean on the subject, map that loobean to an address (using opcode 4) to pick one of two formulas, and run only the chosen one. The combinatory core we have built here is the selection half; the predicate half is a partial motivation for opcodes 3, 4, and 5 as minimal additions to the `SKI` basis rather than merely conveniences.

10 Conclusion

The `SKI` construction in Nock ISA is Turing-complete, but it does not have laziness, normal-form readback, or sharing. Each of these properties marks a boundary where the fragment is genuinely insufficient and additional tooling beyond {0, 1, 2} is required.

This article makes concrete the claim that Nock opcodes 0,

[6] Setting the unchosen branch to a crashing formula confirms it never runs: `IF TRUE 111 ⊥` returns `111` without crashing, `IF FALSE ⊥ 222` returns `222`, and the control case `IF TRUE ⊥ 222` does crash. The chosen branch really executes, so the laziness is not an artifact of dead code.

1, and 2 are sufficient to represent the SKI combinator calculus and are thus Turing-complete (if insufficient to handle nouns natively). The embedding is a constructive proof that the opcode 0, 1, 2 fragment is Turing-complete: the deferred monograph the *Documentary History* expected is rooted in a handful of lines of Nock. The compiler is a homomorphism from SKI application to Nock evaluation, under a fixed call-by-value convention. The combinators are pure construction; opcode 2 appears only in the formula they assemble at application time, carried as quoted data. A second corollary locates the fragment's edge precisely: a conditional's *selection* is combinatory and lives in 0, 1, 2, but its *decision*—computing a boolean from data—does not, which motivates Nock's basis as an SKI core plus a minimal inspection and arithmetic kernel. The Nock ISA has been manifestly Turing-complete since its inception, but this exposition makes a new demonstration explicit and constructive.

References

Curry, Haskell B. and Robert Feys (1958). *Combinatory Logic.* Amsterdam: North-Holland Publishing Company.

Kiselyov, Oleg (2018). "λ to SKI, Semantically: Declarative Pearl." In: *Functional and Logic Programming*. Ed. by John P. Gallagher and Martin Sulzmann. Cham: Springer International Publishing, pp. 33–50.

~lagrev-nocfep, N. E. Davis and Curtis Yarvin ~sorreg-namtyv (2025). "A Documentary History of the Nock Combinator Calculus." In: *Urbit Systems Technical Journal* 2.1, pp. 155–190. URL: https://urbitsystems.tech/article/v02-i01/a-documentary-history-of-the-nock-combinator-calculus.

Smullyan, Raymond M. (1994). *To Mock a Mockingbird and Other Logic Puzzles: : Including an Amazing Adventure in Combinatory Logic.* New York: Knopf.